

GitHub link : <https://github.com/dharnesh13600/lab4-DS.git>

INTRODUCTION:

In this lab I have created a microservice application and deployed in Kubernetes using modern distributed techniques. This includes delving into the topics of designing service boundaries, implementing inter-service communication strategies and considering deployment implications.

I have built an Image Processing System with two services.

Image API service (FastAPI):

This API accepts an image from the user and this original image is stored in the Redis by the API and pushes this job into the Redis Queue. This service also has endpoints to check the status of the processing image. There is another endpoint which saves the processed image.

Image Worker Service (FastAPI):

This service gets the job from the Redis Queue. This service is responsible for applying the greyscale filter, adding a watermark (LAB-4) and changing the size of the image. The processed image is then stored in the Redis and then the status is updated.

Technologies used:

1. Python FastAPI for the microservices.
2. Redis as a message broker.
3. Docker for containerization.
4. Minikube for orchestration (External access via NodePort).

SYSTEM DESIGN & SETUP

Microservice Architecture

The whole application is divided into two microservices

1. The first service does is write heavy component which handles image uploads, queue operations etc.
2. The second service will do the CPU-heavy image processing

Therefore, splitting the system into 2 microservices provides high computational isolation as because they are separate.

ARCHITECTURE FLOW

Step 1: User uploads an image, that is the original image which will be stored in the Redis by the API.

Step 2: API pushes the job (i.e. image_id, filename) onto the Redis list using LPUSH.

Step 3: Worker service receives the job via blocking pop (BRPOP)

Step 4: The worker does the processing of the image which includes watermark, grayscale and resizing.

Step 5: The processed image is stored in the Redis by the worker.

Step 6: User can use status endpoint to check the status and processed endpoint to get the final image.

ARCHITECTURAL ANALYSIS AND JUSTIFICATION:

I have followed the principle of microservice granularity to split the application into two services (i.e. API, Worker).

Microservice granularity

Granularity Disintegrators:

I have two services where one service image. Api handles I/O and network operations. Hence it is a lightweight service. On the other hand, the worker service performs CPU intensive tasks.

Therefore, splitting them into separate services will help the worker to scale independently.

Code Volatility:

The image worker service has high volatility because in future there may be chances of adding new logics or features like adding new image effects, modifying how resizing of image works and even for tuning the performance. So, these updates only affect only image worker microservice not the image API

Whereas Image API will have stable endpoints and the implementation is rarely changed hence it has low volatility.

Performance Isolation

Out of two services the image worker does the CPU heavy tasks and the image API microservice is lightweight. The main purpose of isolating these services because no matter the worker service being busy the image-api service which is user facing should be fast and responsive.

Granularity Integrators

We can see both the services are using the same image ID and both can access the original, processed images and the metadata so which means the components should be in a single service to avoid duplicate access patterns. However, I mitigated this issue by using Redis as a shared storage and messaging layer. Redis is present between the services and acts as a

buffer where it allows components to communicate with the data. Even though the service shares the same data it is not strong to have them in a single service.

Another integrator is transactional boundaries. Uploading and processing an image is not a single transaction because processing is happening asynchronously and eventual consistency is good for this application.

So, from the above integrators I decided that neither shared data and transactional constraints did not justify to merge the services into one. Hence it is a two-service architecture.

Inter Service Communication

I chose asynchronous messaging using Redis queue rather than synchronous REST because of the following:

- ➔ Even though the worker is down API can get uploads.
- ➔ Resilient system
- ➔ Loose coupling
- ➔ Supports horizontal scaling

Trade-Offs

1. For better scalability use of a message broker is necessary and we have used Redis.
2. Improved fault tolerance is achieved by increasing the operational complexity.
3. We have eventual consistency because of decoupling between the services.

I have used Redis queue instead of Rabbit MQ and Kafka because it is lightweight, easy to deploy and the most important feature is the BRPOP where it provides simple blocking consumption.

Architectural Decision Records (ADR):

ADR 1: Python FastAPI

Context: I needed a lightweight, easy to deploy and a fast web framework for creating the microservice.

Decision: FastAPI for both API & Worker

Consequences:

- It has built in swagger
- High performance
- Async support

ADR 2: Redis Message Broker

Context: A asynchronous queue between image API service and the image worker service.

Decision: Redis Lists (LPUSH + BRPOP)

Consequences:

- Simple setup
- Auto failover by blocking the pop

ADR 3: K8s Minikube for Deployment

Context: For orchestration and deployment

Decision: Deploying in Kubernetes

Consequences:

- Auto healing
- Auto scaling
- Used In modern cloud architecture

Screenshots and diagrams

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
image-worker	latest	77b7a9dd4fd5	33 minutes ago	175MB
image-api	latest	c8e70879e1ac	34 minutes ago	188MB

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:49734/upload-image' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'file@lab-4 test 2.jpg;type=image/jpeg'
```

Request URL

http://127.0.0.1:49734/upload-image

Server response

Code Details

200

Response body

```
{
  "image_id": "ebfa6e3f-2e30-4155-92dc-1a156f7c3118",
  "status": "pending"
}
```

Response headers

```
content-length: 70
content-type: application/json
date: Mon, 24 Nov 2025 20:13:55 GMT
server: uvicorn
```

Code Details

200

Response body

```
{
  "image_id": "ebfa6e3f-2e30-4155-92dc-1a156f7c3118",
  "status": "processed",
  "error": null
}
```

Response headers

```
content-length: 85
content-type: application/json
date: Mon, 24 Nov 2025 20:14:39 GMT
server: uvicorn
```

Code Details

200

Response body



Response headers

```
content-length: 58416
content-type: image/png
date: Mon, 24 Nov 2025 20:14:42 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Input image



Output image:

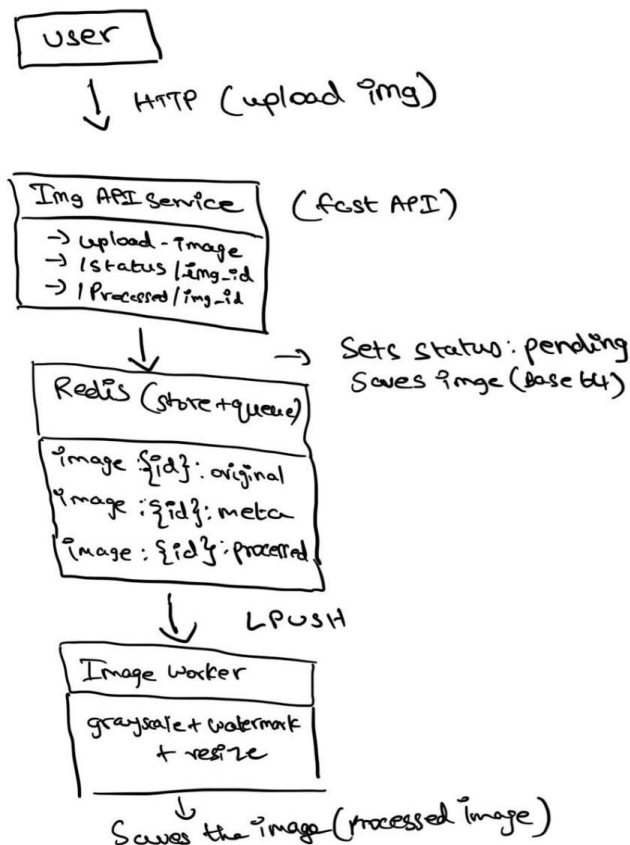


```

PS D:\UCD\Distributed Systems\comp41720-lab4> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
image-api-7bdf8748bd-qr4m8          1/1     Running   0           33h
image-worker-8dd7ddd45-1nng2        1/1     Running   0           9h
redis-6c6fcd64b8-b8d5p             1/1     Running   0           33h
PS D:\UCD\Distributed Systems\comp41720-lab4> kubectl get svc
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
backend-service                    ClusterIP            10.103.211.228   <none>        5000/TCP         17d
image-api-service                   NodePort             10.110.99.145    <none>        8000:30080/TCP   33h
image-worker-service                ClusterIP            10.103.68.237    <none>        8001/TCP         33h
kubernetes                         ClusterIP            10.96.0.1        <none>        443/TCP          17d
redis-service                       ClusterIP            10.110.146.186   <none>        6379/TCP         33h
PS D:\UCD\Distributed Systems\comp41720-lab4>

```

Basic Architecture:



Conclusion:

This lab provided me the hands-on experience in designing and deploying microservices successfully in k8s. I had challenges while doing this lab where the debugging in k8s particularly the Nodeport access. Overall, the lab helped me understand the microservice architecture and the DevOps workflow.

References:

<https://redis.io/docs/latest/commands/brpop/>

<https://pillow.readthedocs.io/en/stable/>

<https://stackoverflow.com/questions/70710874/how-to-send-base64-image-using-python-requests-and-fastapi>

<https://annacsmedeiros.medium.com/efficient-image-processing-in-python-a-straightforward-guide-to-base64-and-numpy-conversions-e9e3aac13312>

<https://codesignal.com/learn/courses/building-an-image-generation-service-with-fastapi/lessons/introduction-to-the-image-manager-with-fastapi>